
Software Design and Architecture

*By Habtam D.
AASTU, 2022*

Chapter Five

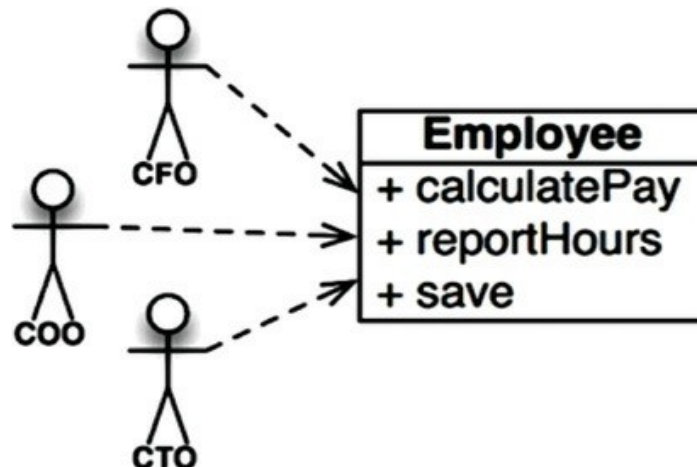
SOLID Principles

Introduction

- Good software systems begin with **clean code**.
- The SOLID principles tell us **how to arrange our functions and data structures** into classes, and how those classes should be interconnected.
- The goal of the principles is the creation of mid-level software structures that:
 - » Tolerate change,
 - » Are easy to understand, and
 - » Are the basis of components that can be used in many software systems.
- SOLID
 - » S – The Single Responsibility Principle
 - » O – The Open-Closed Principle
 - » L – The Liskov Substitution Principle
 - » I – The Interface Segregation Principle
 - » D – The Dependency Inversion Principle

SRP: The Single Responsibility Principle

- “*Every class should have a single responsibility, and that responsibility should be entirely encapsulated by that class*”
- If you think of Responsibility as a reason for change, a class should only have one reason to change, because it only does one thing.
- This responsibility should be entirely encapsulated by that class. In other words: do something once & Don't Repeat Yourself (DRY).



SRP: The Single Responsibility Principle

- Difficulty in naming classes can be a sign that you are trying to do too much, and that this class is taking on too much responsibility.
- Do One Thing: methods that only do one thing are usually very short: 3-5 lines.
- Multiple unit tests for the same method should be a warning sign that you are doing too much.
- When violated: fragile design that breaks in unexpected ways when changed.

OCP: The Open-Closed Principle

- “*Software entities (classes, modules, functions, etc.) should be open for extension, but closed for modification.*”
- **Open for extension** – you can extend an object’s behavior.
- **Closed for modification** – without the need to modify any existing code.
- The ***goal*** is to make the system easy to extend without incurring a high impact of change.
- This can be accomplished by ***partitioning*** the system into components, and arranging those components into a dependency hierarchy that protects higher-level components from changes in lower-level components.

OCP: The Open-Closed Principle

- Change behavior without changing code?
 - » Rely on abstractions, not implementations
 - » Do not limit the variety of implementations
- Violations
 - » Each change requires re-testing (possible bugs)
 - » Cascading changes through modules
 - » Logic depends on conditional statements
- Solution
 - » Using new classes
- Approaches to achieve OCP
 - » Using parameters (pass delegates)
 - » Inheritance (Template method pattern)
 - » Compositions (Strategy pattern)

OCP: The Open-Closed Principle

- OCP add complexity to a design
- No design can be closed against all changes.

LSP: The Liskov Substitution Principle

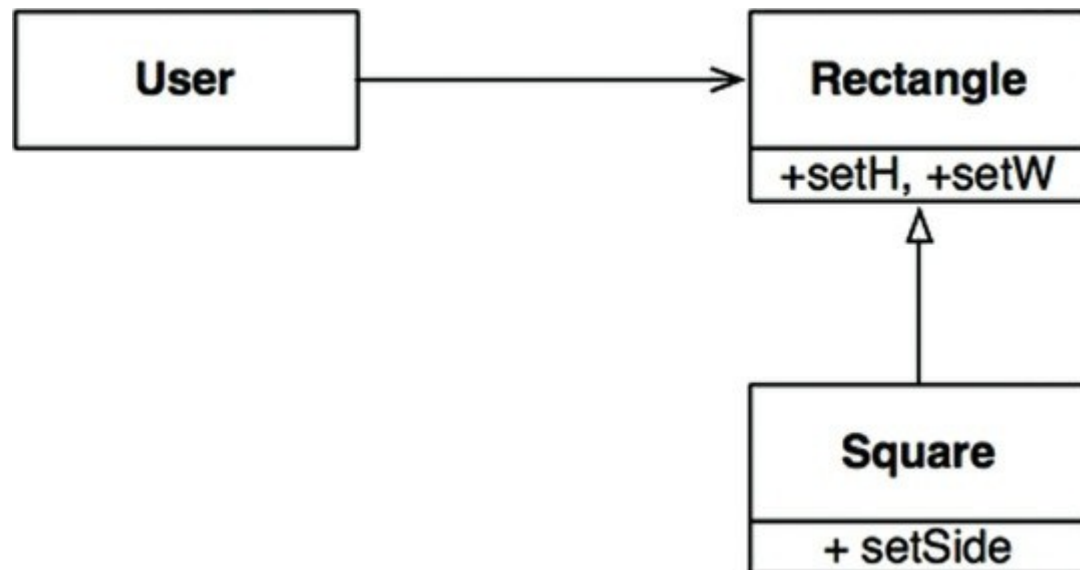
- *“If S is a subtype of T , then objects of type T may be replaced with objects of type S without altering any of the desirable properties of that program”*
- Subtypes must be substitutable for their base types.
- A subclass should only override the parent class' methods in a way that does not break functionality from a client's point of view.
- If Child is derived from Parent, then Parent can be replaced by Child and the program will continue to work as normal.

LSP: The Liskov Substitution Principle

- Favor Interfaces over inheritance
- Use inheritance sparingly.
- Keep inheritance hierarchies from getting to big – aim for no more than two or three deep.

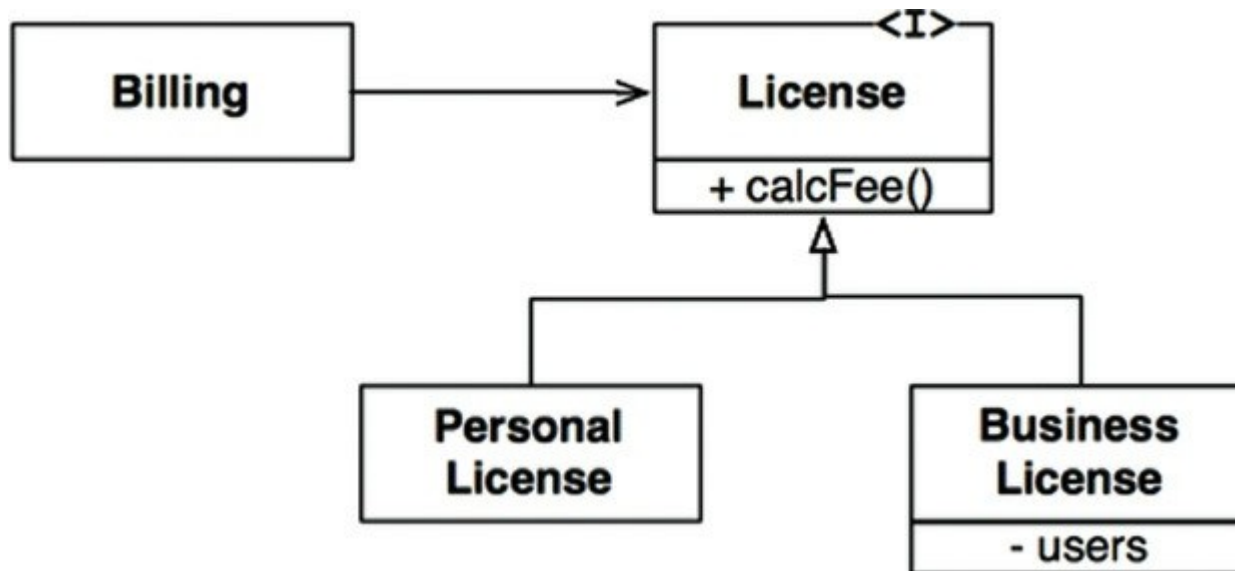
LSP: The Liskov Substitution Principle

- Favor Interfaces over inheritance
- Use inheritance sparingly.
- Keep inheritance hierarchies from getting too big – aim for no more than two or three deep.
- Example: The infamous square/rectangle problem



LSP: The Liskov Substitution Principle

- This design conforms to the LSP because the behavior of the Billing application does not depend, in any way, on which of the two subtypes it uses.
- Both of the subtypes are substitutable for the License type.



ISP: The Interface Segregation Principle

- *“No client should be forced to depend on methods it does not use.”*
- Keep interfaces as small as possible, to avoid unnecessary dependencies.
- Ideally, it should be possible to understand any part of the code in isolation, without needing to look up the rest of the system code.
- Therefore, make fine grained interfaces that are client specific.

ISP: The Interface Segregation Principle

- **Example**

- » Let's look at the following interface: IVehicle
- » At first glance it seems fine... nothing bad going on here.
- » Or is there?

```
public interface IVehicle
{
    void Accelerate();
    void Break();
    void ChangeGear(int newGear);
    void TurnOnRadio();
    void TurnOffRadio();
    void TurnOnHeatedSeats();
    void TurnOffHeatedSeats();
}
```

ISP: The Interface Segregation Principle

- Lets think about when we start building concrete implementations of this interface.
- A Car class would be ok, as it can meet the contract set by the interface...But there is going to be a lot of code here – clearly we are going too much! (Breaking SRP!)
- And what about a Motorbike, or a Bus class? Neither of these need to implement all the methods required by the contract!

```
public class Car : IVehicle
{
    public void Accelerate() { }
    public void Break() { }
    public void ChangeGear(int newGear) { }
    public void TurnOnRadio() { }
    public void TurnOffRadio() { }
    public void TurnOnHeatedSeats() { }
    public void TurnOffHeatedSeats() { }
}

public class MotorBike
{
    public void Accelerate() { }
    public void Break() { }
    public void ChangeGear(int newGear) { }
}

public class Bus
{
    public void Accelerate() { }
    public void Break() { }
    public void ChangeGear(int newGear) { }
}
```

ISP: The Interface Segregation Principle

- We could just throw an exception whenever someone calls the methods we don't need?
- No – this is dirty! This code smells!
- It's a waste of time & effort, and creates lots of exception handling issues for all consumers of the class.

```
public class MotorBike : IVehicle
{
    public void Accelerate() { }
    public void Break() { }
    public void ChangeGear(int newGear) { }

    public void TurnOnRadio() {
        throw new NotImplementedException();}
    public void TurnOffRadio() {
        throw new NotImplementedException();}
    public void TurnOnHeatedSeats() {
        throw new NotImplementedException();}
    public void TurnOffHeatedSeats(){
        throw new NotImplementedException();}
}
```

ISP: The Interface Segregation Principle

- A better solution would be to break the large interface down into smaller, more specific interfaces, based on responsibility.
- Each interface is only concerned with one responsibility (SRP – only has one reason to change).
- Because a single class can implement multiple interfaces, we are now able to pick and choose which functionality we wish to implement in each concrete implementation and we don't need to implement functionality we don't need in order to conform to a contract.

```
public interface IVehicle
{
    void Accelerate();
    void Break();
    void ChangeGear(int newGear);
}
```

```
public interface IAudioControl
{
    void TurnOnRadio();
    void TurnOffRadio();
}
```

```
public interface IHeatedSeats
{
    void TurnOnHeatedSeats();
    void TurnOffHeatedSeats();
}
```

```
public class Car : IVehicle, IAudioControl, IHeatedSeats
{
    public void Accelerate() { }
    public void Break() { }
    public void ChangeGear(int newGear) { }
    public void TurnOnRadio() { }
    public void TurnOffRadio() { }
    public void TurnOnHeatedSeats() { }
    public void TurnOffHeatedSeats() { }
}
```

```
public class MotorBike : IVehicle
{
    public void Accelerate() { }
    public void Break() { }
    public void ChangeGear(int newGear) { }
}
```

```
public class Bus
{
    public void Accelerate() { }
    public void Break() { }
    public void ChangeGear(int newGear) { }
}
```

DIP: The Dependency Inversion Principle

- “A. *High-level modules should not depend on low-level modules. Both should depend on abstractions.*
- B. *Abstractions should not depend on details. Details should depend on abstractions.*”
- High level modules should:
 - » depend upon abstraction of low level modules
 - » force low level modules to change
- When violated: lower level module changes can force high level modules to change.
- Instead of having concrete implementations communicate directly (and depend on each other), decouple them by formalizing their communication interface as an abstract interface based on the needs of the higher-level class.